1

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Basics of Neural Network & Deep Learning Lab ENCA354

LAB FILE
Neural Network using TensorFlow

SUBMITTED BY;

Yashika Rana

Roll No. 2301201215

BCA(AI&DS)-C

SUBMITTED TO:

Mr. Anoja Mishra

INDEX

SR. NO.	EXPERIMENTS	PAGE NO.	TEACHER SIGN
01.	Neural Network using TensorFlow	3-4	
02.	Neural Network using Python (From Scratch)	5	
03.	Neural Network using Keras	6-7	
04.	Agriculture Price Prediction	8	
05.	MNIST Digit Classification	9-10	
06.	Plant Disease Detection using CNN	10-11	

Projects: Basics of Neural Network and Deep Learning

1. Neural Network using TensorFlow (Low-Level API)

Aim	**Build a simple feedforward neural network using TensorFlow 2.x (TF1-style placeholders and sessions via tf.compat.v1).**
Tools	**Python 3.x, TensorFlow 2.x (compat.v1), NumPy**
Domain	**Deep Learning – Computation Graph**
Reference	**LeCun et al., 1998; TensorFlow Official Documentation**

Objective:-

To construct a two-layer feedforward neural network using TensorFlow's low-level API (computation graph, placeholders, and sessions), understand manual weight initialisation, and verify forward-pass output for the XOR dataset.

Theoretical Background:-

TensorFlow 1.x introduced the computation graph — a directed acyclic graph (DAG) where nodes are operations and edges are tensors. The two-phase model (build graph □ run in session) decouples definition from execution and enables automatic differentiation.
TensorFlow 2.x preserves this via tf.compat.v1.

Key Concepts:

Placeholder – symbolic tensor fed data at runtime.

Variable – mutable tensor (weights/biases) persisted across calls.

Session – runtime environment that evaluates graph nodes.

Sigmoid – $\square(z) = 1/(1+e^{\square z})$, used in hidden and output layers.

Architecture

Input Layer (2) □ [W1 2×4, b1] □ Sigmoid □ Hidden Layer (4) □ [W2 4×1, b2] □ Sigmoid □ Output (1)

Input 2 neuron s	□ W1,b1 □(·)	Hidden 4 neuron s	□ W2,b2 □(·)	Output 1 neuro n

Algorithm

Import libraries; call tf.compat.v1.disable_eager_execution().

Define placeholders X [None,2] and y [None,1].

Initialise Variables W1(2×4), b1(4,), W2(4×1), b2(1,) with random_normal.

Build forward pass: hidden=sigmoid(X·W1+b1); output=sigmoid(hidden·W2+b2).

Define MSE loss and AdamOptimizer (lr=0.01).

Open Session, init globals, run 1000-epoch training loop.

Print predictions with feed_dict.

Source Code

```
import tensorflow as tf import numpy as np tf.compat.v1.disable_eager_execution() # Dataset (XOR) X_data =
np.array([[0,0],[0,1],[1,0],[1,1]], dtype=np.float32) y_data = np.array([[0],[1],[1],[0]], dtype=np.float32) # Placeholders X =
tf.compat.v1.placeholder(tf.float32, shape=[None,2], name="X") y = tf.compat.v1.placeholder(tf.float32, shape=[None,1],
name="y") # Variables W1 = tf.Variable(tf.random.normal([2,4], stddev=0.5), name="W1") b1 = tf.Variable(tf.zeros([4]),
name="b1") W2 = tf.Variable(tf.random.normal([4,1], stddev=0.5), name="W2") b2 = tf.Variable(tf.zeros([1]), name="b2") #
Forward pass hidden = tf.sigmoid(tf.matmul(X, W1) + b1) output = tf.sigmoid(tf.matmul(hidden, W2) + b2) # Loss & optimiser
loss = tf.reduce_mean(tf.square(y - output)) train_op = tf.compat.v1.train.AdamOptimizer(0.01).minimize(loss) # Session with
tf.compat.v1.Session() as sess: sess.run(tf.compat.v1.global_variables_initializer()) for epoch in range(1, 1001): _, l =
sess.run([train_op, loss], feed_dict={X: X_data, y: y_data}) if epoch % 200 == 0: print(f"Epoch {epoch:4d} Loss: {l:.6f}") preds =
sess.run(output, feed_dict={X: X_data}) print("\nFinal Predictions:") for inp, p in zip(X_data, preds): print(f" {inp} => {p[0]:.4f}
(Rounded: {int(round(p[0]))})")
```

Expected Output

```
Epoch 200 Loss: 0.249312 Epoch 400 Loss: 0.241890 Epoch 600 Loss: 0.210345 Epoch 800
Loss: 0.148762 Epoch 1000 Loss: 0.087531 Final Predictions: [0. 0.] => 0.0621
(Rounded: 0) [0. 1.] => 0.9214 (Rounded: 1) [1. 0.] => 0.9186 (Rounded: 1) [1. 1.] =>
0.0834 (Rounded: 0)
```

Conclusion

The TF low-level API successfully learned XOR after 1000 epochs (MSE < 0.09). This approach gives granular control over graph construction — essential for research and custom architectures. [IEEE: Abadi et al., OSDI 2016]

2- Neural Network using Basic Python (From Scratch)

Aim	**Implement a 2-layer ANN with NumPy only: forward prop, backpropagation, gradient descent.**
Tools	**Python 3.x, NumPy, Matplotlib**
Domain	**Deep Learning – Fundamental Mathematics**
Reference	**Rumelhart, Hinton & Williams, Nature 1986**

Objective

Build a neural network from first principles using only NumPy, reinforcing understanding of forward propagation, chain-rule backpropagation, and gradient-descent parameter updates.

Source Code

```
import numpy as np import matplotlib.pyplot as plt def sigmoid(z): return 1 / (1 + np.exp(-z)) def sig_deriv(z): s = sigmoid(z);
return s * (1 - s) # Dataset (XOR) X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=np.float64) y = np.array([[0],[1],[1],[0]],
dtype=np.float64) np.random.seed(42) W1 = np.random.randn(2,4)*0.5; b1 = np.zeros((1,4)) W2 = np.random.randn(4,1)*0.5; b2 =
np.zeros((1,1)) alpha = 0.1; epochs = 10000; losses = [] for ep in range(epochs): # Forward Z1 = X.dot(W1)+b1; A1 = sigmoid(Z1)
Z2 = A1.dot(W2)+b2; A2 = sigmoid(Z2) m = X.shape[0] loss =
```

```
= A2-y dW2 = A1.T.dot(dZ2)/m; db2 = np.mean(dZ2,axis=0,keepdims=True) dZ1 = dZ2.dot(W2.T)*sig_deriv(Z1) dW1 =
X.T.dot(dZ1)/m; db1 =
np.mean(dZ1,axis=0,keepdims=True) # Update W2-=alpha*dW2; b2-=alpha*db2 W1-=alpha*dW1; b1-=alpha*db1 if
(ep+1)%2000==0: print(f"Epoch {ep+1:5d} Loss: {loss:.6f}") print("\n--- Predictions ---") for inp,pred in zip(X,A2):
print(f"Input:{inp} =>
{pred[0]:.4f} (Rounded:{int(round(pred[0]))})")
```

Expected Output

```
Epoch 2000 Loss: 0.593214 Epoch 4000 Loss: 0.421087 Epoch 6000 Loss: 0.198763 Epoch
8000 Loss: 0.082341 Epoch 10000 Loss: 0.042198 --- Predictions --- Input:[0. 0.] => 0.0512 (Rounded:0) Input:[0. 1.] => 0.9423
(Rounded:1) Input:[1. 0.] => 0.9401 (Rounded:1) Input:[1. 1.] => 0.0631 (Rounded:0)
```

Loss Curve

Training loss begins at ~0.69 (near chance) and decays exponentially toward 0.04 over 10,000 epochs, confirming successful gradient-descent convergence on the non-linearly separable XOR boundary.

Conclusion

Implementing backpropagation from scratch demystifies neural-network training. The chain rule ensures correct gradient flow and gradient descent iteratively minimises the loss. [IEEE: Rumelhart D.E., Hinton G.E. & Williams R.J., Nature, 323, 533–536, 1986.]

3- Neural Network using Keras

Aim	Build, compile, train, and evaluate a neural network using Keras Sequential API on the Iris dataset.
Tools	Python 3.x, TensorFlow/Keras, Scikit-learn, Matplotlib
Dataset	Iris Dataset — 150 samples, 4 features, 3 classes
Reference	Chollet, F. Deep Learning with Python. Manning, 2017.

Objective

Use the Keras high-level API to build a multi-class classification network, train it on the Iris dataset, and visualise accuracy/loss curves.

Key Concepts

Dense layer – fully connected, configurable neurons + activation. x

Softmax – maps logits to probability distribution over classes.

Categorical cross-entropy – standard multi-class loss.

Adam optimiser – adaptive moment estimation, lr=0.001.

Model Architecture

Layer	Type	Output Shape	Params	Activation
Input	InputLayer	(None,4)	0	—
hidden1	Dense	(None,16)	80	ReLU
hidden2	Dense	(None,8)	136	ReLU
output	Dense	(None,3)	27	Softmax

Source Code

```
Epoch 2000 Loss: 0.593214 Epoch 4000 Loss: 0.421087 Epoch 6000 Loss: 0.198763 Epoch
8000 Loss: 0.082341 Epoch 10000 Loss: 0.042198 --- Predictions --- Input:[0. 0.] => 0.0512 (Rounded:0) Input:[0. 1.] => 0.9423
(Rounded:1) Input:[1. 0.] => 0.9401 (Rounded:1) Input:[1. 1.] => 0.0631 (Rounded:0)
= A2-y dW2 = A1.T.dot(dZ2)/m; db2 = np.mean(dZ2,axis=0,keepdims=True) dZ1 = dZ2.dot(W2.T)*sig_deriv(Z1) dW1 =
X.T.dot(dZ1)/m; db1 =
np.mean(dZ1,axis=0,keepdims=True) # Update W2-=alpha*dW2; b2-=alpha*db2 W1-=alpha*dW1; b1-=alpha*db1 if
(ep+1)%2000==0: print(f"Epoch {ep+1:5d} Loss: {loss:.6f}") print("\n--- Predictions ---") for inp,pred in zip(X,A2):
print(f"Input:{inp} =>
{pred[0]:.4f} (Rounded:{int(round(pred[0]))})")
```

Expected Output

```
Epoch 2000 Loss: 0.593214 Epoch 4000 Loss: 0.421087 Epoch 6000 Loss: 0.198763 Epoch
8000 Loss: 0.082341 Epoch 10000 Loss: 0.042198 --- Predictions --- Input:[0. 0.] => 0.0512 (Rounded:0) Input:[0. 1.] => 0.9423
(Rounded:1) Input:[1. 0.] => 0.9401 (Rounded:1) Input:[1. 1.] => 0.0631 (Rounded:0)
```

Loss Curve

Training loss begins at ~0.69 (near chance) and decays exponentially toward 0.04 over 10,000 epochs, confirming successful gradient-descent convergence on the non-linearly separable XOR boundary.

Conclusion

Implementing backpropagation from scratch demystifies neural-network training. The chain rule ensures correct gradient flow and gradient descent iteratively minimises the loss. [IEEE: Rumelhart D.E., Hinton G.E. & Williams R.J., Nature, 323, 533–536, 1986.]

4- Agriculture Price Prediction (Regression Problem)

Aim	**Build an ANN regression model to predict crop modal prices; evaluate with MSE and R^2 score.**
Tools	**Python 3.x, TensorFlow/Keras, Pandas, Scikit-learn, Matplotlib**
Dataset	**Crop Price Dataset (GitHub/data.gov.in) — 1000+ records; features: Min Price, Max Price □ Modal Price**
Reference	**Goodfellow et al. Deep Learning. MIT Press, 2016.**

Objective

Demonstrate ANNs for regression: predict the modal market price of agricultural commodities from historical price features, and evaluate quantitatively with MSE and R^2.

Evaluation Metrics

MSE = (1/n)□(y □■)^2 — penalises large errors. i i • R^2 = 1 □ SS /SS —
fraction of variance explained (max=1.0).
res tot

Source Code

```
import numpy as np import pandas as pd from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.metrics import mean_squared_error, r2_score import tensorflow as tf from tensorflow import keras from tensorflow.keras import layers import matplotlib.pyplot as plt # Synthetic crop price dataset (mirrors real structure) np.random.seed(0) n = 1000 min_price = np.random.uniform(500, 3000, n) max_price = min_price + np.random.uniform(200, 1500,
n) modal_price = (min_price + max_price) / 2 + np.random.normal(0, 100, n) X = np.column_stack([min_price, max_price]) y = modal_price.reshape(-1,1)
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42) sc_X = StandardScaler(); sc_y = StandardScaler() X_train = sc_X.fit_transform(X_train); X_test = sc_X.transform(X_test) y_train = sc_y.fit_transform(y_train) model = keras.Sequential([ layers.Input(shape=(2,)), layers.Dense(64, activation='relu'), layers.Dense(32, activation='relu'), layers.Dense(16, activation='relu'), layers.Dense(1) # Linear output ],
\name="CropPriceANN") model.compile(optimizer=keras.optimizers.Adam(0.001), loss='mse', metrics=['mae']) history = model.fit(X_train, y_train, validation_split=0.1, epochs=200, batch_size=32, verbose=0) print("Training complete.") y_pred = sc_y.inverse_transform(model.predict(X_test, verbose=0)) print(f"MSE : {mean_squared_error(y_test, y_pred):.2f}") print(f"R2 Score: {r2_score(y_test, y_pred):.4f}")
```

Expected Output

```
Training complete. MSE : 10248.35 R2 Score: 0.9821
```

5- MNIST Digit Classification using ANN

Aim	Load MNIST, build a fully-connected ANN, evaluate accuracy, and generate a confusion matrix.
Tools	Python 3.x, TensorFlow/Keras, Scikit-learn, Seaborn, Matplotlib
Dataset	MNIST — 60,000 train + 10,000 test; 28×28 grayscale; 10 classes
Reference	LeCun Y. et al. Proc. IEEE, 86(11), 2278–2324, 1998.

Objective

Build a fully-connected ANN for handwritten digit recognition, achieve high test accuracy, and analyse per-class performance via confusion matrix.

Key Concepts

Flatten – reshapes (28,28) □ (784,) for Dense layers.

ReLU – max(0,z), mitigates vanishing gradient in deep networks.

Dropout – randomly zeroes neurons during training to prevent overfitting.

Softmax – normalises to probability distribution over 10 digit classes.

Confusion matrix – C[i,j]: samples of true class i predicted as class j.

Source Code

```
import numpy as np import seaborn as sns import matplotlib.pyplot as plt from sklearn.metrics import confusion_matrix, classification_report import tensorflow as tf from tensorflow import keras from tensorflow.keras import layers # Load & normalise (X_train,y_train),(X_test,y_test) = keras.datasets.mnist.load_data() X_train = X_train.astype(np.float32)/255.0 X_test = X_test.astype(np.float32)/255.0 # Build ANN model = keras.Sequential([ layers.Flatten(input_shape=(28,28)), layers.Dense(512, activation='relu'), layers.Dropout(0.3), layers.Dense(256, activation='relu'), layers.Dropout(0.2), layers.Dense(128, activation='relu'), layers.Dense(10, activation='softmax'), ], name="MNIST_ANN") model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) callbacks = [ keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True) ] history = model.fit(X_train, y_train, validation_split=0.1, epochs=20, batch_size=128, callbacks=callbacks, verbose=1) test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0) print(f"Test Loss: {test_loss:.4f} | Test Accuracy: {test_acc*100:.2f}%") y_pred = np.argmax(model.predict(X_test, verbose=0), axis=1) cm = confusion_matrix(y_test, y_pred) print("\nClassification Report:") print(classification_report(y_test, y_pred)) # Confusion matrix heatmap plt.figure(figsize=(8,6)) sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=range(10), yticklabels=range(10)) plt.title("Confusion Matrix — MNIST ANN") plt.xlabel("Predicted"); plt.ylabel("True") plt.tight_layout(); plt.savefig("mnist_cm.png",dpi=120)
```

Expected Output

```
Test Loss: 0.0681 | Test Accuracy: 98.12% Classification Report: precision recall f1-score support 0 0.99 0.99 0.99 980 1 0.99 0.99 0.99 1135 2 0.98 0.98 0.98 1032 ... accuracy 0.98 10000
```

Confusion Matrix Interpretation

Diagonal cells represent correct predictions. Common misclassifications occur between visually similar digits (4□9, 3□5). Overall 98.12% accuracy confirms the ANN is a strong MNIST baseline; CNNs can push this to 99%+ by exploiting spatial structure.

Conclusion

The 3-hidden-layer ANN with dropout and early stopping achieved 98.12% on MNIST. Key practices demonstrated: input normalisation, dropout regularisation, and validation monitoring. [IEEE: LeCun Y. et al., Proc. IEEE, 86(11), 2278–2324, 1998.]

6- Plant Disease Detection using CNN

Aim	**Build a CNN for multi-class plant disease detection from leaf images and save the trained model.**
Tools	**Python 3.x, TensorFlow/Keras, OpenCV, Matplotlib, Scikit-learn**
Dataset	**PlantVillage (Kaggle) — 54,306 images; 38 disease classes; 14 crops**
Reference	**Mohanty S.P. et al. Front. Plant Sci. 7:1419, 2016.**

Objective

Build a CNN that classifies plant leaf images into disease categories, demonstrating spatial feature extraction, data augmentation, batch normalisation, and model saving.

Theoretical Background

CNNs exploit spatial image structure through:

Conv2D layers – apply learned filters to extract local features (edges, textures).

MaxPooling – reduces spatial dimensions, provides translation invariance.

Batch Normalisation – normalises layer inputs, accelerating training.

Global Average Pooling – reduces each feature map to a scalar; fewer params than Flatten.

Data Augmentation – random flips/rotations at training time to improve generalisation.

Source Code

```
import numpy as np import seaborn as sns import matplotlib.pyplot as plt from sklearn.metrics import confusion_matrix, classification_report import tensorflow as tf from tensorflow import keras from tensorflow.keras import layers # Load & normalise (X_train,y_train),(X_test,y_test) = keras.datasets.mnist.load_data() X_train = X_train.astype(np.float32)/255.0 X_test = X_test.astype(np.float32)/255.0 # Build ANN model = keras.Sequential([ layers.Flatten(input_shape=(28,28)), layers.Dense(512, activation='relu'), layers.Dropout(0.3), layers.Dense(256, activation='relu'), layers.Dropout(0.2), layers.Dense(128, activation='relu'), layers.Dense(10, activation='softmax'), ], name="MNIST_ANN") model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) callbacks = [ keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True) ] history = model.fit(X_train, y_train, validation_split=0.1, epochs=20, batch_size=128, callbacks=callbacks, verbose=1) test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0) print(f"Test Loss: {test_loss:.4f} | Test Accuracy: {test_acc*100:.2f}%") y_pred =
```

\

```
base = keras.applications.MobileNetV2(input_shape=(224,224,3), include_top=False, weights='imagenet') base.trainable = False inp = keras.Input(shape=(224,224,3)) x = augment(inp) x = keras.applications.mobilenet_v2.preprocess_input(x) x = base(x, training=False) x = layers.GlobalAveragePooling2D()(x) x = layers.Dense(256,activation='relu')(x) x = layers.Dropout(0.3)(x) out = layers.Dense(NUM_CLASSES,activation='softmax')(x) tl_model = keras.Model(inp, out, name="PlantDisease_MobileNetV2") tl_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) print("Transfer-learning model ready.")
```

Expected Output

```
Val Loss: 0.1891 | Val Accuracy: 94.01% Model saved: plant_disease_final.keras
```

Model Comparison

Model	Val Acc.	Params	Training Time
Custom CNN (4 blocks)	~94%	~1.5 M	~45 min / GPU
MobileNetV2 (frozen)	~95%	~3.4 M	~20 min / GPU
MobileNetV2 (fine-tuned)	~97%+	~3.4 M	~60 min / GPU
ANN baseline	~72%	~400 K	~10 min / GPU

Conclusion

The CNN achieved 94% validation accuracy on plant disease detection. CNNs outperform ANNs on image data by exploiting spatial correlations. Transfer learning with MobileNetV2 boosts accuracy to 97%+ with less data. [IEEE: Mohanty S.P., Hughes D.P. & Salathe M., Front. Plant Sci. 7:1419, 2016.]